Лабораторна робота № 3

Тема. Робота з великими мовними моделями через API.

Основи Prompt Engineering

Мета роботи: Ознайомитися з принципами взаємодії з великими мовними моделями (LLM), навчитися формулювати запити до мовних моделей та досліджувати вплив структури prompt на отриманий результат.

Використана модель: gemini-3.5-flash-lite

Завдання

Завдання 1. Взаємодія з великою мовною моделлю Обрати будь-яку доступну велику мовну модель (наприклад, ChatGPT, Gemini, Claude або іншу) та виконати 5 різних типів запитів:

1. Пояснення — попросити пояснити певне поняття.

2. Генерація тексту — створити невеликий текст на задану тему.

3. Переклад — перекласти текст з однієї мови на іншу.

4. Узагальнення — скоротити та узагальнити запропонований текст.

5. Генерація списку — сформувати список за заданими критеріями.

```
from google import genai

client = genai.Client(api_key="API-KEY")

tasks = {
    "1. Пояснення": "Поясни простими словами, що таке рекурсія в програмуванні.",
    "2. Генерація тексту": "Напиши невеликий текст (до 5 речень) про те, яку мову програмування краще вивчити новачку.",
    "3. Переклад": "Переклади французькою мовою терміни: 'змінна', 'цикл з умовою', 'рекурсія', 'об'єктно-орієнтоване програмування'.",
    "4. Узагальнення": "Узагальни однією фразою текст: 'Об'єктно-орієнтоване програмування (ООП) — це методологія програмування, яка базується на концепції об'єктів, що містять дані у вигляді полів та код у вигляді методів, що дозволяє значно спростити підтримку та масштабування великих програмних систем.'",
    "5. Генерація списку": "Сформуй список із 4 найпопулярніших середовищ розробки (IDE) або редакторів коду для програмування з коротким описом для кожного(2 речення)."
}
```

```
for task_name, prompt_text in tasks.items():
    print(f"{task_name}")
    print(f"PROMPT: {prompt_text}\n\n")

    interaction = client.interactions.create(
        model="gemini-3.5-flash-lite",
        input=prompt_text
    )

    print("Відповідь:")
    print(interaction.output_text)
```

1. Пояснення

PROMPT: Поясни простими словами, що таке рекурсія в програмуванні.

Відповідь:

Рекурсія в програмуванні — це коли функція викликає саму себе всередині свого ж тіла.

Якщо пояснювати зовсім просто, це як **ефект дзеркала в дзеркалі**: одне дзеркало дивиться в інше, і там утворюється нескінченна галерея зменшених копій самого себе.

Класичний приклад із життя: Матрьошка

Уяви українську матрьошку:

1. Ти відкриваєш велику матрьошку.

2. Всередині лежить менша.

3. Ти відкриваєш меншу, і всередині лежить ще одна (теж матрьошка!).

4. Ти робиш одну і ту саму дію (відкриваєш матрьошку), поки не дійдеш до найменшої, цільної ляльки, яку вже не можна відкрити.

Ось цей процес відкривання однакових предметів один за одним — це і є рекурсія.

Як це виглядає в коді?

У рекурсії обов'язково мають бути дві частини:

1. **Умова виходу (базовий випадок)** — момент, коли треба зупинитися, інакше програма зависне назавжди (як та матрьошка, яка не відкривається).

2. **Крок рекурсії** — дія, де функція викликає саму себе, але з трохи меншою чи простішою задачею.

Приклад (рахунок до нуля):

Уяви, що треба відрахувати від 3 до 0:

* Функція каже: «Дай мені число 3».

* 3 — це не нуль? Тоді виводимо 3 і викликаємо цю ж функцію для числа **2**.

* Для числа 2: виводимо 2 і викликаємо функцію для **1**.

* Для числа 1: виводимо 1 і викликаємо функцію для **0**.

* Дійшли до 0? Це умова виходу! Зупиняємося.

Чому рекурсію люблять і бояться?

* **Чому люблять:** Вона робить код дуже коротким і красивим для складних завдань (наприклад, пошук у глибину в лабіринті чи робота з деревами папок на комп'ютері).

* **Чому бояться:** Якщо забути про «умову виходу», програма буде викликати себе нескінченно, доки у комп'ютера не закінчиться пам'ять (це називається *Stack Overflow*).

Коротко кажучи: **рекурсія — це спосіб вирішити велику задачу, розбиваючи її на такі самі, але менші задачі.**

---

2. Генерація тексту

PROMPT: Напиши невеликий текст (до 5 речень) про те, яку мову програмування краще вивчити новачку.

Відповідь:

Для новачків найкращим вибором зазвичай стає **Python**, адже його простий синтаксис нагадує звичайну англійську мову і легко сприймається. Ця мова є надзвичайно універсальною — від створення простих скриптів до штучного інтелекту та веб-розробки. Крім того, Python має величезну спільноту початківців, тому знайти відповідь на будь-яке запитання в інтернеті дуже легко. Якщо ж вам цікавіше одразу створювати сайти, гарною альтернативою стане вивчення зв'язки **HTML, CSS та JavaScript**. Головне — обрати один напрямок і не боятися робити перші практичні кроки.

---

3. Переклад

PROMPT: Переклади французькою мовою терміни: 'змінна', 'цикл з умовою', 'рекурсія', 'об'єктно-орієнтоване програмування'.

Відповідь:

Ось переклад запропонованих програмістських термінів французькою мовою:

* **змінна** — *une variable*

* **цикл з умовою** — *une boucle conditionnelle* (також часто використовують назви конкретних циклю цього типу: *boucle while* або *boucle tant que*)

* **рекурсія** — *la récursion* (або *la récursivité*)

* **об'єктно-орієнтоване програмування** — *la programmation orientée objet* (скорочено **POO**)

---

4. Узагальнення

PROMPT: Узагальни однією фразою текст: 'Об'єктно-орієнтоване програмування (ООП) — це методологія програмування, яка базується на концепції об'єктів, що містять дані у вигляді полів та код у вигляді методів, що дозволяє значно спростити підтримку та масштабування великих програмних систем.'

Відповідь:

Об'єктно-орієнтоване програмування — це методологія створення програм на основі об'єктів (даних і методів), яка спрощує їхню підтримку та масштабування.

---

5. Генерація списку

PROMPT: Сформуй список із 4 найпопулярніших середовищ розробки (IDE) або редакторів коду для програмування з коротким описом для кожного(2 речення).

Відповідь:

Ось список із 4 найпопулярніших середовищ розробки та редакторів коду:

1. **Visual Studio Code (VS Code)**

Універсальний та надзвичайно популярний редактор коду від Microsoft з відкритим кодом. Завдяки величезній екосистемі плагінів він легко адаптується практично для будь-якої мови програмування та налаштовується під будь-які потреби розробника.

2. **IntelliJ IDEA**

Потужне середовище розробки від компанії JetBrains, яке вважається золотим стандартом для Java-розробників. Воно пропонує надзвичайно розумний аналіз коду, автодоповнення та глибоку інтеграцію з різноманітними фреймворками й інструментами.

3. **PyCharm**

Спеціалізоване середовище розробки від JetBrains, створене спеціально для мови Python. Воно містить усе необхідне для ефективної веб-розробки, аналізу даних та роботи з масовим машинним навчанням "з коробки".

4. **Visual Studio**

Повноцінне професійне середовище розробки від Microsoft, призначене переважно для створення складних застосунків на C#, C++ та .NET. Воно пропонує потужні вбудовані інструменти для дебагінгу, тестування та командної роботи над великими проєктами.

№	Тип завдання	Prompt	Результат	Аналіз
1	Пояснення	Поясни простими словами, що таке рекурсія в програмуванні.	**Рекурсія** в програмуванні — це коли функція викликає саму себе всередині свого ж тіла. Якщо пояснювати зовсім просто, це як **ефект дзеркала в дзеркалі**: одне дзеркало дивиться в інше, і там утворюється нескінченна галерея зменшених копій самого себе. ### Класичний приклад із життя: Матрьошка Уяви українську матрьошку: 1. Ти відкриваєш велику матрьошку. 2. Всередині лежить менша. 3. Ти відкриваєш меншу, і всередині лежить ще одна (теж матрьошка!). 4. Ти робиш одну і ту саму дію (відкриваєш матрьошку), поки не дійдеш до найменшої, цільної ляльки, яку вже не можна відкрити. Ось цей процес відкривання однакових предметів один за одним — це і є рекурсія. ### Як це виглядає в коді? У рекурсії обов'язково мають бути дві частини: 1. **Умова виходу (базовий випадок)** — момент, коли треба зупинитися, інакше програма	Відповідь структурована, легко сприймається початківцем, містить життєві аналогії та попередження про stack overflow.

			зависне назавжди (як та матрьошка, яка не відкривається). 2. **Крок рекурсії** — дія, де функція викликає саму себе, але з трохи меншою чи простішою задачею. **Приклад (рахунок до нуля):** Уяви, що треба відрахувати від 3 до 0: * Функція каже: «Дай мені число 3». * 3 — це не нуль? Тоді виводимо 3 і викликаємо цю ж функцію для числа **2**. * Для числа 2: виводимо 2 і викликаємо функцію для **1**. * Для числа 1: виводимо 1 і викликаємо функцію для **0**. * Дійшли до 0? Це умова виходу! Зупиняємося. ### Чому рекурсію люблять і бояться? * **Чому люблять:** Вона робить код дуже коротким і красивим для складних завдань (наприклад, пошук у глибину в лабіринті чи робота з деревами папок на комп'ютері). * **Чому бояться:** Якщо забути про «умову виходу», програма буде викликати себе нескінченно, доки у комп'ютера не закінчиться пам'ять (це називається *Stack Overflow*). Коротко кажучи: **рекурсія — це спосіб вирішити велику задачу, розбиваючи її на такі самі, але менші задачі.**	
2	Генерація тексту	Напиши невеликий текст (до 5 речень) про те, яку мову програмування краще вивчити новачку.	Для новачків найкращим вибором зазвичай стає **Python**, адже його простий синтаксис нагадує звичайну англійську мову і легко сприймається. Ця мова є надзвичайно універсальною — від створення простих скриптів до штучного інтелекту та веб-розробки. Крім того, Python має величезну спільноту початківців, тому знайти відповідь на будь-яке запитання в інтернеті дуже легко. Якщо ж вам цікавіше одразу створювати сайти, гарною альтернативою стане вивчення зв'язки **HTML, CSS та JavaScript**. Головне — обрати один напрямок і не боятися робити перші практичні кроки.	Модель чітко дотрималася обмеження по довжині, надала логічні аргументи та зберегла зв'язність тексту.
3	Переклад	Переклади французькою мовою терміни: 'змінна', 'цикл з умовою', 'рекурсія', 'об'єктно-орієнтоване програмування'.	Ось переклад запропонованих програмістських термінів французькою мовою: * **змінна** — *une variable* * **цикл з умовою** — *une boucle conditionnelle* (також часто використовують назви конкретних циклю цього типу: *boucle while* або *boucle tant que*) * **рекурсія** — *la récursion* (або *la récursivité*) * **об'єктно-орієнтоване програмування** — *la programmation orientée objet* (скорочено **POO**)	Точний термінологічний переклад з урахуванням специфіки програмування та додаванням загальноприйнятих абревіатур.
4	Узагальнення	Узагальни однією фразою текст: 'Об'єктно-орієнтоване програмування (ООП) — це методологія програмування, яка базується на концепції об'єктів...	Об'єктно-орієнтоване програмування — це методологія створення програм на основі об'єктів (даних і методів), яка спрощує їхню підтримку та масштабування.	Модель точно виділила головну частину вихідного тексту і вклалася у суворе обмеження «в одну фразу»
5	Генерація списку	Сформуй список із 4 найпопулярніших	Ось список із 4 найпопулярніших середовищ розробки та редакторів коду: 1. **Visual Studio Code (VS Code)** Універсальний та надзвичайно популярний редактор коду від Microsoft з відкритим кодом.	Суворе дотримання формату

		середовищ розробки (IDE) або редакторів коду для програмування з коротким описом для кожного(2 речення).	Завдяки величезній екосистемі плагінів він легко адаптується практично для будь-якої мови програмування та налаштовується під будь-які потреби розробника. 2. **IntelliJ IDEA** Потужне середовище розробки від компанії JetBrains, яке вважається золотим стандартом для Java-розробників. Воно пропонує надзвичайно розумний аналіз коду, автодоповнення та глибоку інтеграцію з різноманітними фреймворками й інструментами. 3. **PyCharm** Спеціалізоване середовище розробки від JetBrains, створене спеціально для мови Python. Воно містить усе необхідне для ефективної веб-розробки, аналізу даних та роботи з масовим машинним навчанням "з коробки". 4. **Visual Studio** Повноцінне професійне середовище розробки від Microsoft, призначене переважно для створення складних застосунків на C#, C++ та .NET. Воно пропонує потужні вбудовані інструменти для дебагінгу, тестування та командної роботи над великими проєктами.	нумерованого списку та ліміту речень для кожного пункту.

Завдання 2. Порівняння способів формування prompt

Для однієї обраної задачі створити два варіанти prompt:

1. Zero-shot prompt — сформулювати завдання без прикладів.
2. Few-shot prompt — перед виконанням основного завдання надати моделі 2–3 приклади бажаного результату.

```
from google import genai


client = genai.Client(api_key="API-KEY")

zero_shot_input = f"""
Визнач мову програмування для наступного фрагмента коду та коротко опиши його
призначення:
print(list(map(lambda x: x**2, range(5))))
"""

print("ZERO-SHOT PROMPT")
print(zero_shot_input.strip(), "\n\n")

interaction_zero = client.interactions.create(
    model="gemini-3.5-flash-lite",
    input=zero_shot_input.strip()
)
print("Відповідь (Zero-shot):")
print(interaction_zero.output_text, "\n")

few_shot_input = f"""
Визнач мову програмування для фрагмента коду та опиши його призначення у такому ж
форматі, як у прикладах нижче.
```

```
Приклад 1:
Код: console.log("Hello, World!");
Мова: JavaScript
Опис: Виводить текстове повідомлення у консоль.

Приклад 2:
Код: System.out.println("Test");
Мова: Java
Опис: Здійснює виведення рядка тексту в стандартний потік виведення.

Основне завдання:
Код: print(list(map(lambda x: x**2, range(5))))
Мова:
"""

print("FEW-SHOT PROMPT")
print(few_shot_input.strip(), "\n\n")

interaction_few = client.interactions.create(
    model="gemini-3.5-flash-lite",
    input=few_shot_input.strip()
)
print("Відповідь (Few-shot):")
print(interaction_few.output_text)
```

ZERO-SHOT PROMPT

Визнач мову програмування для наступного фрагмента коду та коротко опиши його призначення:

print(list(map(lambda x: x**2, range(5))))

Відповідь (Zero-shot):

Мова програмування: Python.

Призначення коду:

Цей код створює список квадратів цілих чисел від 0 до 4 включно і виводить його на екран.

Результат виконання коду: `[0, 1, 4, 9, 16]`

Короткий розбір того, як це працює:

* `range(5)` генерує послідовність чисел від 0 до 4 (`0, 1, 2, 3, 4`).

* `lambda x: x**2` — це коротка анонімна функція, яка підносить передане число `x` до квадрата.

* `map(...)` застосовує цю функцію до кожного елемента з `range(5)`.

* `list(...)` перетворює отриманий результат у звичайний список і виводить його за допомогою `print()`.

FEW-SHOT PROMPT

Визнач мову програмування для фрагмента коду та опиши його призначення у такому ж форматі, як у прикладах нижче.

Приклад 1:

Код: console.log("Hello, World!");

Мова: JavaScript

Опис: Виводить текстове повідомлення у консоль.

Приклад 2:

Код: System.out.println("Test");

Мова: Java

Опис: Здійснює виведення рядка тексту в стандартний потік виведення.

Основне завдання:

Код: print(list(map(lambda x: x**2, range(5))))

Мова:

Відповідь (Few-shot):

Python

Опис: Створює список квадратів чисел від 0 до 4 включно та виводить його на екран.

Підхід	Prompt	Результат	Висновок
Zero-shot	Визнач мову програмування для наступного фрагмента коду та коротко опиши його призначення: print(list(map(lambda x: x**2, range(5))))	**Мова програмування:** Python. **Призначення коду:** Цей код створює список квадратів цілих чисел від 0 до 4 включно і виводить його на екран. Результат виконання коду: `[0, 1, 4, 9, 16]` **Короткий розбір того, як це працює:** * `range(5)` генерує послідовність чисел від 0 до 4 (`0, 1, 2, 3, 4`). * `lambda x: x**2` — це коротка анонімна функція, яка підносить передане число `x` до квадрата. * `map(...)` застосовує цю функцію до кожного елемента з `range(5)`. * `list(...)` перетворює отриманий результат у звичайний список і виводить його за допомогою `print()`.	Результат детальний та навчальний, але не дотримується лаконічної форми, оскільки форматування не було задане прикладами.
Few-shot	Визнач мову програмування для фрагмента коду та опиши його призначення у такому ж форматі, як у прикладах нижче. Приклад 1: Код: console.log("Hello, World!"); Мова: JavaScript Опис: Виводить текстове повідомлення у консоль. Приклад 2:	Python Опис: Створює список квадратів чисел від 0 до 4 включно та виводить його на екран.	Результат відповідає наданим прикладам та є коротким і локанічним

	Код: System.out.println("Test"); Мова: Java Опис: Здійснює виведення рядка тексту в стандартний потік виведення. Основне завдання: Код: print(list(map(lambda x: x**2, range(5)))) Мова:		

Завдання 3. Удосконалення prompt

Отримати відповідь моделі на простий або недостатньо конкретний prompt.

Наприклад: Розкажи про штучний інтелект.

Проаналізувати недоліки такого prompt та створити його вдосконалений варіант.

Під час вдосконалення можна додати: • роль моделі; • конкретну інструкцію; • контекст; • бажаний обсяг відповіді; • формат відповіді; • цільову аудиторію. Порівняти результати виконання початкового та вдосконаленого prompt.

```
from google import genai


client = genai.Client(api_key="API-KEY")

print("Простий PROMPT")
print("Розкажи про об'єктно-орієнтоване програмування.\n")

interaction_initial = client.interactions.create(
    model="gemini-3.5-flash-lite",
    input="Розкажи про об'єктно-орієнтоване програмування."
)
print("Відповідь (Простий):")
print(interaction_initial.output_text, "\n")

improved_prompt = """
Ти — Senior Python-розробник і ментор. Поясни поняття «Об'єктно-орієнтоване
програмування (ООП)» для студента, який тільки починає вивчати програмування.
Структуруй відповідь у вигляді двох блоків:
1. Суть ООП простими словами;
2. Перелічи та стисло поясни 3 основні принципи (інкапсуляція, успадкування,
поліморфізм).
Використовуй дружній та зрозумілий стиль.
"""
```

```
print("Вдосконалений PROMPT ")
print(improved_prompt.strip(),"\n\n")

interaction_improved = client.interactions.create(
    model="gemini-3.5-flash-lite",
    input=improved_prompt.strip()
)
print("Відповідь (Вдосконалений):")
print(interaction_improved.output_text)
```

Простий PROMPT

Розкажи про об'єктно-орієнтоване програмування.

Відповідь (Простий):

Об'єктно-орієнтоване програмування (ООП) — це методологія програмування, в якій основною концепцією є поняття **об'єктів**, які об'єднують у собі дані (властивості) та поведінку (методи).

Якщо традиційне (процедурне) програмування фокусується на «діях» (функціях), то ООП фокусується на «сутностях» (об'єктах), якими ми маніпулюємо.

Давай розберемося з цією концепцією детально.

---

1. Основні поняття ООП

Уся концепція ООП тримається на двох китах: **класах** та **об'єктах**.

* **Клас (Class)** — це креслення, шаблон або опис. Він визначає, які характеристики (властивості) та дії (методи) матиме майбутній об'єкт.

  * *Приклад:* Клас `Кіт`. У нього є властивості (колір, вік, вага) і методи (нявкати, спати, їсти).

* **Об'єкт (Object)** — це конкретний екземпляр класу.

  * *Приклад:* Мій рудий кіт Барсік, якому 3 роки і який важить 4 кг. Він створений за шаблоном класу `Кіт`.

---

2. Чотири стовпи ООП

ООП базується на чотирьох головних принципах:

A. Інкапсуляція (Encapsulation)

Це приховування внутрішньої реалізації об'єкта та захист даних. Ми об'єднуємо дані та методи, які з ними працюють, в один клас, а доступ до деяких деталей робимо обмеженим (використовуємо модифікатори доступу на кшталт `private`, `public`).

* *Аналогія:* Автомобіль. Ви знаєте, як натискати на педаль газу (зовнішній інтерфейс), але вам не потрібно знати, як саме подається паливо в циліндри двигуна (інкапсульована логіка).

Б. Наслідування (Inheritance)

Це здатність одного класу успадковувати властивості та методи іншого класу. Це дозволяє уникнути дублювання коду: загальну логіку виносять у батьківський клас, а дочірні класи її доповнюють або змінюють.

*   *Аналогія:* Є клас `Транспорт` (має колеса і швидкість). Класи `Автомобіль` і `Велосипед` можуть успадкувати ці властивості, додавши свої специфічні (кількість дверей або наявність педалей).

В. Поліморфізм (Polymorphism)

Це можливість різних об'єктів реагувати по-різному на той самий виклик (метод). Тобто, один і той самий інтерфейс може мати різну реалізацію.

*   *Аналогія:* У вас є команда «Подати голос». Якщо її дати коту, він скаже «Няу», якщо собаці — «Гав», а якщо качці — «Кря». Метод однаковий (`speak()`), але результат різний.

Г. Абстракція (Abstraction)

Це виділення головних, найважливіших характеристик об'єкта і відкидання другорядних. Абстракція допомагає працювати з ідеями на високому рівні, не занурюючись у дрібні деталі.

*   *Аналогія:* Коли ви пишете програму для лікарні, людина як `Пацієнт` матиме історію хвороби та групу крові, але її колір очей чи улюблена музика програмі не цікаві.

---

3. Переваги ООП

1. **Модульність:** Код легко розбити на окремі незалежні частини (класи). Над різними класами можуть працювати різні розробники.

2. **Повторне використання коду (Reusability):** Завдяки наслідуванню не потрібно писати один і той самий код двічі.

3. **Легкість підтримки та масштабування:** Якщо треба змінити логіку роботи програми, зазвичай достатньо змінити один клас, не чіпаючи інші.

4. **Безпека даних:** Інкапсуляція захищає дані від випадкових пошкоджень з інших частин програми.

---

4. Недоліки ООП

*   **Складність:** Для простих завдань ООП може бути «зброєю масового ураження». Написання кількох класів для виведення «Hello, World» — це надмірно.

*   **Витрати пам'яті:** Об'єкти та абстракції іноді вимагають більше системних ресурсів, ніж прості процедурні програми.

*   **Поріг вході:** Новачкам часто важко зрозуміти концепції на кшталт поліморфізму чи ієрархії класів.

Де використовується ООП?

ООП є стандартом у більшості сучасних мов програмування. Головні мови, що підтримують ООП:

* **Java**

* **C++ / C#**

* **Python**

* **JavaScript** (має прототипне ООП, але підтримує класи)

* **PHP**

* **Swift / Kotlin**

Короткий підсумок: ООП — це спосіб організувати хаос у коді, представивши програму у вигляді взаємодіючих між собою «реальних» об'єктів. Це робить код зрозумілішим, більшим за об'ємом, але набагато легшим для командної розробки та підтримки.

Вдосконалений PROMPT

Ти — Senior Python-розробник і ментор. Поясни поняття «Об'єктно-орієнтоване програмування (ООП)» для студента, який тільки починає вивчати програмування.

Структуруй відповідь у вигляді двох блоків:

1. Суть ООП простими словами;

2. Перелічи та стисло поясни 3 основні принципи (інкапсуляція, успадкування, поліморфізм).

Використовуй дружній та зрозумілий стиль.

Відповідь (Вдосконалений):

Привіт! Радий вітати тебе на шляху до вивчення Python. Як ментор скажу відверто: початківці часто лякаються абревіатури «ООП», але насправді це просто зручний спосіб організувати твій код так, щоб він був схожий на реальний світ.

Давай розберемося без заумних термінів з Вікіпедії.

---

1. Суть ООП простими словами

Уяви, що ти пишеш гру. Тобі потрібно створити багато різних персонажів: воїнів, магів, монстрів.

У класичному програмуванні (процедурному) тобі довелося б створювати окремі змінні для кожного героя (наприклад, `warrior_hp`, `mage_hp`) і купу функцій для управління ними. Це швидко перетворюється на хаос.

ООП пропонує інший підхід. Ми дивимося на світ як на сукупність **об'єктів**.

* **Клас (Class)** — це креслення або рецепт. Наприклад, креслення «Кіт». За цим кресленням ми можемо створити скільки завгодно конкретних котів.

* **Об'єкт (Object)** — це вже готовий виріб за цим кресленням. Наприклад, твій рудий кіт Барсик і сусідський чорний кіт Мурчик.

У кожного об'єкта є дві речі:

1. **Дані (атрибути)** — характеристики (колір, вік, ім'я). У Python це зазвичай змінні всередині класу.

2. **Поведінка (методи)** — те, що об'єкт може робити (м'ячати, спати, їсти). У Python це функції всередині класу.

Тобто ООП допомагає запакувати дані та логику їх обробки всередину одного «контейнера» (об'єкта), щоб код був чистим і зрозумілим.

---

2. 3 основні принципи ООП

У фундаменті ООП лежать три кити. Давай пройдемося по них на простих прикладах:

1. Інкапсуляція (Capsule / Капсула)

* **Суть:** Це принцип об'єднання даних та методів, які з ними працюють, в одному класі, а також **захист** цих даних від несанкціонованого втручання ззовні.

* **На прикладі:** У тебе є мікрохвильовка. Усередині неї є складні схеми, магнетрон тощо (це закрита капсула). Тобі не треба знати, як вона працює всередині — ти просто натискаєш кнопку «Старт» зовні. Більше того, виробник захистив тебе від випадкового удару струмом.

* *У Python:* Ми можемо приховувати певні змінні об'єкта, щоб інші частини програми не могли їх випадково зламати чи змінити (використовуємо символи підкреслення, наприклад, `__speed`).

2. Успадкування (Inheritance)

* **Суть:** Можливість створювати новий клас на основі вже існуючого. Новий клас успадковує всі властивості та методи старого, і до них можна додати нові або змінити старі. Це рятує нас від написання однакового коду двічі (DRY — Don't Repeat Yourself).

* **На прикладі:** У тебе є загальний клас `ТранспортнийЗасіб` (у нього є колеса, швидкість, він може їхати). Тобі не потрібно писати все це з нуля для класу `Автомобіль` чи `Велосипед`. Ти просто кажеш: «Автомобіль — це транспортний засіб, але з чотирма дверима та кермом».

* *У Python:* Ми пишемо `class Car(Vehicle):`, і всі функції батьківського класу автоматично стають доступними для дочірнього.

3. Поліморфізм (Poly — багато, morph — форма)

* **Суть:** Дослівно — «багато форм». Це принцип, коли різні об'єкти можуть мати однакові назви методів, але виконувати їх по-різному.

* **На прикладі:** Візьмемо команду «Видай звук».

  * Кіт скаже: «Мяу!»

  * Собака скаже: «Гав!»

  * Качка скаже: «Кря!»

  Команда одна й та сама (`make_sound()`), але кожен об'єкт реалізує її по-своєму. Тобі не потрібно запам'ятовувати окрему функцію `dog_sound()` і `cat_sound()` — ти просто викликаєш один метод для всіх.

* *У Python:* Це дозволяє писати універсальний код, який працює з різними типами об'єктів однаково легко.

Не хвилюйся, якщо спочатку це здається абстрактним. Як твій ментор, обіцяю: щойно ми напишеш перші кілька власних класів у Python, усе стане на свої місця. Готовий спробувати написати першого кота в коді? ☺

Підхід	Prompt	Результат	Висновок
Початковий prompt	Розкажи про об'єктно-орієнтоване програмування.	Відповідь (Простий): **Об'єктно-орієнтоване програмування (ООП)** — це методологія програмування, в якій основною концепцією є поняття **об'єктів**, які об'єднують у собі дані (властивості) та поведінку (методи). Якщо традиційне (процедурне) програмування фокусується на «діях» (функціях), то ООП фокусується на «сутностях» (об'єктах), якими ми маніпулюємо. Давай розберемося з цією концепцією детально. --- ### 1. Основні поняття ООП Уся концепція ООП тримається на двох китах: **класах** та **об'єктах**. * **Клас (Class)** — це креслення, шаблон або опис. Він визначає, які характеристики (властивості) та дії (методи) матиме майбутній об'єкт. * *Приклад:* Клас `Кіт`. У нього є властивості (колір, вік, вага) і методи (нявкати, спати, їсти). * **Об'єкт (Object)** — це конкретний екземпляр класу. * *Приклад:* Мій рудий кіт Барсік, якому 3 роки і який важить 4 кг. Він створений за шаблоном класу `Кіт`. --- ### 2. Чотири стовпи ООП ООП базується на чотирьох головних принципах: #### А. Інкапсуляція (Encapsulation) Це приховування внутрішньої реалізації об'єкта та захист даних. Ми об'єднуємо дані та методи, які з ними працюють, в один клас, а доступ до деяких деталей робимо обмеженим (використовуємо модифікатори доступу на кшталт `private`, `public`). * *Аналогія:* Автомобіль. Ви знаєте, як натискати на педаль газу (зовнішній інтерфейс), але вам не потрібно знати, як саме подається паливо в циліндри двигуна (інкапсульована логіка). #### Б. Наслідування (Inheritance) Це здатність одного класу успадковувати властивості та методи іншого класу. Це дозволяє уникнути дублювання коду: загальну логіку виносять у батьківський клас, а дочірні класи її доповнюють або змінюють. * *Аналогія:* Є клас `Транспорт` (має колеса і швидкість). Класи `Автомобіль` і `Велосипед` можуть успадкувати ці властивості, додавши свої специфічні (кількість дверей або наявність педалей). #### В. Поліморфізм (Polymorphism) Це можливість різних об'єктів реагувати по-різному на той самий виклик (метод). Тобто, один і той самий інтерфейс може мати різну реалізацію. * *Аналогія:* У вас є команда «Подати голос». Якщо її дати коту, він скаже «Няу», якщо собаці — «Гав», а якщо качці —	Prompt занадто загальний. Відповідь виявилася перевантаженою деталями, які користувач не запитував.

		«Кря». Метод однаковий (`speak()`), але результат різний. #### Г. Абстракція (Abstraction) Це виділення головних, найважливіших характеристик об'єкта і відкидання другорядних. Абстракція допомагає працювати з ідеями на високому рівні, не занурюючись у дрібні деталі. * *Аналогія:* Коли ви пишете програму для лікарні, людина як `Пацієнт` матиме історію хвороби та групу крові, але її колір очей чи улюблена музика програмі не цікаві. …	
Вдосконалений prompt	Ти — Senior Python-розробник і ментор. Поясни поняття «Об'єктно-орієнтоване програмування (ООП)» для студента, який тільки починає вивчати програмування. Структуруй відповідь у вигляді двох блоків: 1. Суть ООП простими словами; 2. Перелічи та стисло поясни 3 основні принципи (інкапсуляція, успадкування, поліморфізм).	Привіт! Радий вітати тебе на шляху до вивчення Python. Як ментор скажу відверто: початківці часто лякаються абревіатури «ООП», але насправді це просто зручний спосіб організувати твій код так, щоб він був схожий на реальний світ. Давай розберемося без заумних термінів з Вікіпедії. --- ### 1. Суть ООП простими словами Уяви, що ти пишеш гру. Тобі потрібно створити багато різних персонажів: воїнів, магів, монстрів. У класичному програмуванні (процедурному) тобі довелося б створювати окремі змінні для кожного героя (наприклад, `warrior_hp`, `mage_hp`) і купу функцій для управління ними. Це швидко перетворюється на хаос. **ООП пропонує інший підхід.** Ми дивимося на світ як на сукупність **об'єктів**. * **Клас (Class)** — це креслення або рецепт. Наприклад, креслення «Кіт». За цим кресленням ми можемо створити скільки завгодно конкретних котів. * **Об'єкт (Object)** — це вже готовий виріб за цим кресленням. Наприклад, твій рудий кіт Барсик і сусідський чорний кіт Мурчик. У кожного об'єкта є дві речі: 1. **Дані (атрибути)** — характеристики (колір, вік, ім'я). У Python це зазвичай змінні всередині класу. 2. **Поведінка (методи)** — те, що об'єкт може робити (м'ячати, спати, їсти). У Python це функції всередині класу. Тобто ООП допомагає запакувати дані та логику їх обробки всередину одного «контейнера» (об'єкта), щоб код був чистим і зрозумілим. --- ### 2. 3 основні принципи ООП У фундаменті ООП лежать три кити. Давай пройдемося по них на простих прикладах: #### 1. Інкапсуляція (Capsule / Капсула) * **Суть:** Це принцип об'єднання даних та методів, які з ними працюють, в одному класі, а також **захист** цих даних від несанкціонованого втручання ззовні. * **На прикладі:** У тебе є мікрохвильовка. Усередині неї є складні схеми, магнетрон тощо (це закрита капсула). Тобі не треба знати, як вона працює всередині — ти просто натискаєш кнопку «Старт» зовні. Більше того, виробник захистив тебе від випадкового удару струмом. * *У Python:* Ми можемо приховувати певні змінні об'єкта, щоб інші частини програми не могли їх випадково зламати чи змінити (використовуємо символи підкреслення, наприклад, `__speed`). #### 2. Успадкування (Inheritance) * **Суть:** Можливість створювати новий клас на основі вже існуючого. Новий клас успадковує всі властивості та методи	Завдяки додаванню ролі, контексту, цільової аудиторії та суворого формату відповідь стала точковою, структурованою та приємною для читання.

		старого, і до них можна додати нові або змінити старі. Це рятує нас від написання однакового коду двічі (DRY — Don't Repeat Yourself). …	

Завдання 4. Використання системної ролі

Дослідити вплив system role / системної інструкції на відповідь мовної моделі. Для однієї задачі використати різні ролі, наприклад: • викладач; • програміст; • науковець.

```
from google import genai

client = genai.Client(api_key="API-KEY")

user_task = "Поясни, чому важливо правильно обробляти винятки (Exceptions / try-catch) у коді."

roles = {
    "1. Викладач": "Ти — досвідчений та терплячий викладач програмування в університеті. Твоя мета — пояснити матеріал методично, зрозуміло для студентів, використовуючи аналогії та навчальний тон. Обсяг: до 4-5 речень.",

    "2. Програміст-практик": "Ти — Senior Software Engineer з 10-річним досвідом розробки високонавантажених систем. Поясни тему з точки зору продакшену, надійності системи, логування та підтримки коду. Використовуй професійний сленг. Обсяг: до 4-5 речень.",

    "3. Науковець": "Ти — науковець у галузі програмної інженерії та теорії систем. Поясни тему строгою академічною мовою, спираючись на поняття відмовостійкості, надійності ПЗ та теорії систем. Обсяг: до 4-5 речень."
}

print(f"Завдання: {user_task}")

for role_name, system_instruction_text in roles.items():
    print(f"\nРоль: {role_name}")

    interaction = client.interactions.create(
        model="gemini-3.5-flash-lite",
        system_instruction=system_instruction_text,
        input=user_task
    )

    print("Відповідь:")
```

```
    print(interaction.output_text)
```

Завдання: Поясни, чому важливо правильно обробляти винятки (Exceptions / try-catch) у коді.

Роль: 1. Викладач

Відповідь:

Уявіть, що ваш код — це літак, який летить на автопілоті, а виняток — це несподівана турбулентність або зграя птахів на шляху. Якщо не налаштувати систему реагування (`try-catch`), літак просто розіб'ється від найменшої перешкоди, завершивши роботу програми аварійно. Правильна обробка помилок діє як подушка безпеки, що дозволяє м'яко перехопити проблему, повідомити про неї користувача та продовжити політ. Завдяки цьому наша програма залишається надійною, стабільною і не лякає користувачів раптовим закриттям.

Роль: 2. Програміст-практик

Відповідь:

Необроблений exception у production — це прямий шлях до падіння процесу (crash), забивання дисків трейсами в логах та деградації SLA через необроблені таймаути на балансувальнику. Правильний `try-catch` у критичних точках входу (наприклад, на мікросервісних роутах) дозволяє м'яко перехопити помилку, залогувати контекст з кореляційним ID (correlation ID) для зручного дебагу в ELK і повернути клієнту стандартизований статус-код замість 500-ї білої сторінки. Головне — ніколи не робити "голий" `catch (Exception)`, який ховає баги, і не глушити помилки мовчки, щоб не перетворити підтримку системи на нескінченний гам у продакшені.

Роль: 3. Науковець

Відповідь:

З погляду теорії систем та інженерії надійності програмного забезпечення, коректна обробка винятків є фундаментальним механізмом забезпечення відмовостійкості (fault tolerance) та збереження цілісності системи в умовах невизначеності середовища виконання. Ігнорування або неадекватна перехоплення аномалій призводить до каскадних збоїв (cascade failures), неконтрольованого завершення процесів та порушення інваріантів стану системи. Використання конструкцій типу `try-catch` дозволяє локалізувати дефекти, реалізувати стратегії відновлення (graceful degradation) або безпечного аварійного виходу, мінімізуючи час простою та запобігаючи деградації загальної архітектури.

Аналіз

Викладач: Дохідливо пояснює складну концепцію через просту та яскраву аналогію з літаком, допомагаючи новачкові легко зрозуміти призначення обробки помилок.

Програміст-практик: Фокусується на реаліях продакшену та практичних ризиках, оперуючи інженерним сленгом і конкретними інструментами на кшталт ELK та статус-кодів.

Науковець: Описує проблему на фундаментальному рівні системної інженерії, аналізуючи відмовостійкість, каскадні збої та збереження інваріантів стану.